PA2 Evaluación aplicada

Guion de exposición sobre listas enlazadas

Curso	Algoritmo y Estructura de Datos Basados en Inteligencia Artificial
Docente	Wildr Julio Espinoza Bravo
NRC	4134
Integrantes	Leonardo Cuadros Jhonatan Lino Anthony Fernando Victor Sinche
Repositorio	https://github.com/totomanxd321/PA2_Listas_Enlazadas_Grupo5

Video pendiente de grabación y publicación.

Leonardo Cuadros — Introducción, caso y actividad 1 (2:00–2:20)

En pantalla: portada del informe y diagrama `01_lista_simple.png`.

Texto para exponer

> “Buenos días, profesor. Somos el equipo de cuatro integrantes y vamos a sustentar nuestra PA2 del curso de Algoritmo y Estructura de Datos Basados en Inteligencia Artificial. El tema es el uso de listas enlazadas para organizar solicitudes de atención.

El caso nos plantea una organización donde las solicitudes cambian constantemente: llegan nuevas, algunas se eliminan, otras necesitan cambiar de posición, y en ciertos momentos hay que regresar a un caso anterior o revisar todos los casos de manera continua.

Para entender el problema, definimos que cada solicitud tendrá un código, un asunto y un estado. Por ejemplo, S101 corresponde a una constancia, S102 a una orientación y S103 a un reclamo. Cada solicitud se almacena dentro de un nodo.

Una lista enlazada no necesita que todos sus nodos estén guardados en posiciones consecutivas de memoria. Lo que mantiene su orden son los enlaces entre nodos. La lista simple tiene un enlace hacia el siguiente; la doble tiene uno hacia el anterior y otro hacia el siguiente; y la circular conecta el último nodo nuevamente con el primero.

En el diagrama mostramos cinco solicitudes: S101, S102, S103, S104 y S105. La cabeza señala S101 y el siguiente de S105 es nulo. Esta es nuestra estructura inicial y la base para explicar los cambios. Elegimos este ejemplo porque es fácil ver qué sucede con cada enlace cuando se inserta, modifica o retira una solicitud.

Ahora mi compañero explicará las operaciones que realizamos sobre la lista simple.”

Acción: señalar cabeza, cinco nodos y enlace final a `NULO`.

Jhonatan Lino — Actividad 2: recorrido e inserciones (2:10–2:30)

En pantalla: informe sección “Actividad 2”, imágenes `01_lista_simple.png` y `02_insertar_despues.png`, y salida de la demostración.

Texto para exponer

> “Voy a explicar el recorrido completo y las tres formas de inserción que pide la actividad dos.

Primero, para recorrer una lista simple creamos una referencia llamada actual que empieza en la cabeza. Mientras actual no sea nulo, mostramos la solicitud y avanzamos usando actual.siguiente. Cuando llegamos al final, el recorrido termina. En nuestro ejemplo, el orden es S101, S102, S103, S104 y S105.

Para insertar al inicio creamos el nodo S100. Primero hacemos que su enlace siguiente apunte a la antigua cabeza, que era S101, y después actualizamos la cabeza para que apunte a S100. El orden pasa a ser S100, S101, S102, S103, S104 y S105.

Para insertar al final creamos S106. Como guardamos una referencia a la cola, enlazamos la cola anterior con S106 y después actualizamos la cola. No necesitamos recorrer todos los nodos para realizar ese cambio de punteros.

La tercera operación es insertar después de un nodo dado. Por ejemplo, queremos añadir S102A después de S102. Primero buscamos S102 por su código. Una vez encontrado, hacemos que S102A apunte a quien venía después, que era S103. Recién entonces enlazamos S102 con S102A. Así evitamos perder el resto de la lista.

Estas operaciones nos permiten registrar solicitudes en lugares distintos. Algo importante es que buscar el código S102 puede tardar más que conectar los nodos, por eso distinguimos el costo de búsqueda del costo de los enlaces.

Con esto queda explicado el recorrido y las inserciones. Le doy pase a Anthony Fernando.”

Acción: mostrar las tres líneas correspondientes en la terminal y comparar antes/después.

Anthony Fernando — Actividad 3: búsqueda, modificación y eliminación (2:10–2:30)

En pantalla: `03_eliminar_nodo.png`, sección de pseudocódigo, salida `Buscar S103` y `Eliminar S104`.

Texto para exponer

> “En la actividad tres trabajamos con la lista enlazada simple. Definimos que la búsqueda se realizará por el código de solicitud, porque un código nos permite identificar un caso de forma clara.

El algoritmo comienza en la cabeza y compara cada código. Si encuentra el buscado, devuelve el nodo. Si llega al final, devuelve nulo. En nuestro ejemplo buscamos S103, que corresponde a un reclamo.

Una vez localizado el nodo, podemos modificar sus datos sin moverlo. En la demostración cambiamos el estado de S103 de ‘Pendiente’ a ‘En proceso’. Sus enlaces permanecen iguales: solo se cambia un campo del registro.

Para eliminar un nodo necesitamos cuidar la cadena. Utilizamos dos referencias: actual, que apunta al nodo revisado, y anterior, que apunta al nodo previo. Si quitamos S104, hacemos que S103 apunte directamente a S105. El enlace hacia S104 deja de formar parte del recorrido.

También consideramos los casos especiales. Si se elimina la primera solicitud, se actualiza la cabeza. Si se elimina la última, se actualiza la cola. Y si la lista queda vacía, ambas referencias pasan a nulo.

En cuanto a eficiencia, buscar por código es O(n), porque en el peor caso revisamos todos los nodos. Eliminar un nodo después de encontrar a su anterior solo requiere ajustar enlaces, pero la búsqueda previa hace que la eliminación por código pueda ser O(n). Además, para responder al caso, agregamos la reubicación: S105 se mueve después de S101 sin crear una segunda solicitud.

Estas decisiones hacen que el sistema mantenga el orden correcto. Ahora Victor Sinche explicará por qué otras estructuras son útiles en el mismo problema.”

Acción: resaltar la flecha S103 → S105 y ejecutar el ejemplo de modificación.

Victor Sinche — Actividad 4: lista doble y circular (2:10–2:30)

En pantalla: `04_lista_doble.png` y `05_lista_circular.png`.

Texto para exponer

> “Para la actividad cuatro nos preguntamos cuándo una lista simple deja de ser la opción más cómoda.

Nuestra primera situación es la revisión de solicitudes por un trabajador. Supongamos que está en S202 y necesita ir al caso anterior para comparar información. Si usamos una lista simple, no existe el enlace directo hacia atrás. Tendría que recorrer nuevamente desde la cabeza hasta ubicar el nodo anterior. Por eso proponemos una lista doble.

En esta estructura cada nodo guarda dos referencias: anterior y siguiente. Nuestro ejemplo es S201, S202 y S203. Podemos mostrar el recorrido S201, S202, S203, pero también el inverso: S203, S202, S201. La ventaja es la navegación bidireccional; la desventaja es que debemos mantener más enlaces y usar algo más de memoria.

Nuestra segunda situación es una ronda de revisión periódica. Una vez que se revisa S301, luego S302 y después S303, el proceso debe regresar a S301. Para eso usamos una lista circular simple. El enlace del último nodo ya no apunta a nulo, sino que regresa al primer nodo.

Esto evita reiniciar manualmente el recorrido en cada ronda. Sin embargo, hay que tener cuidado: como no existe un enlace nulo al final, un ciclo sin condición de salida podría continuar indefinidamente. Por eso la demostración usa un contador de nodos para una vuelta y un cursor que avanza en cada turno.

No elegimos una sola estructura para todo porque cada una responde a una necesidad distinta: organización básica, navegación hacia atrás o revisión continua. A continuación presentaré la comprobación de la solución.”

Acción: recorrer los tres nodos hacia ambos lados y luego simular cinco turnos circulares.

Victor Sinche — Implementación, pruebas y cierre (2:20–3:00)

En pantalla: árbol del repositorio, `app/estructuras.py`, terminal y README.

Texto para exponer

> “Para comprobar que nuestra propuesta funciona, desarrollamos un programa pequeño en Python. No usamos las listas de Python como almacenamiento principal de estas estructuras: creamos clases de nodos y referencias que conectan unos nodos con otros.

En la carpeta app está la implementación de la lista simple, doble y circular. En demo.py está la demostración del caso. Voy a ejecutar python -m app.demo. Primero aparecen las cinco solicitudes. Luego se ve cómo insertamos S100 al inicio, S106 al final y S102A después de S102. También se ve que buscamos y modificamos S103, eliminamos S104 y reubicamos S105 después de S101. Al final comprobamos que la lista doble se recorre en ambos sentidos y que la circular vuelve al primer turno después del tercero.

Además, agregamos pruebas automáticas. Ahora ejecutamos python -m unittest discover -s tests -v. Las trece pruebas sirven para verificar, entre otras cosas, que al borrar la cabeza o la cola no se pierdan referencias, que no se permitan códigos repetidos en la lista simple y que la lista circular conserve su ciclo.

En el README explicamos qué problema resuelve el proyecto, quiénes participaron, cómo están distribuidos los archivos, cómo ejecutar el ejemplo y dónde encontrar el informe. El repositorio está disponible en https://github.com/totomanxd321/PA2_Listas_Enlazadas_Grupo5. Después de publicar el video, añadiremos su enlace al README.

En conclusión, la lista simple es suficiente para administrar solicitudes en una secuencia, la doble mejora la navegación en ambos sentidos y la circular permite repetir la revisión por turnos. Aprendimos que elegir una estructura no depende solo de que funcione, sino también de las operaciones que requiere el problema y del costo de mantener sus enlaces.

Muchas gracias. Quedamos atentos a sus preguntas.”

Acción: ejecutar ambos comandos en vivo; mostrar el resultado real de 13 pruebas satisfactorias.

Preguntas que podría hacer el docente

¿Por qué no usar siempre una lista doble? Porque la lista simple necesita menos enlaces por nodo y es suficiente cuando solo se recorre hacia adelante. La doble se justifica cuando retroceder es frecuente.

¿Qué sucede si se elimina la cabeza? La referencia cabeza pasa al segundo nodo; si era el único, cabeza y cola quedan nulos.

¿Por qué buscar por código es O(n)? Porque los nodos no permiten acceso directo por índice; en el peor caso se revisan todos.

¿Insertar al final es O(1) en su código? Cambiar los enlaces sí, gracias al puntero cola, pero el programa revisa antes si el código está repetido; esa validación hace que la llamada completa sea O(n).

¿Cómo evitan un bucle infinito en la lista circular? La operación una_vuelta() recorre exactamente tamano nodos. El cursor de turnos avanza un nodo por cada solicitud de revisión.

¿Qué parte no puede demostrar el código? La asistencia real a clases y la participación del equipo; eso lo registra el docente.